Web Service Network Embedding Based on Link Prediction and Convolutional Learning

Min Shi, *Student Member, IEEE*, Yuan Zhuang, *Member, IEEE*, Yufei Tang, *Member, IEEE*, Maohua Lin, Xingquan Zhu, *Senior Member, IEEE*, and Jianxun Liu

Abstract—Extensive efforts have been applied to develop efficient feature extraction algorithms, which aim to achieve optimal results in many fundamental tasks such as Web-based software service clustering, recommendation and composition. However, one common issue for existing methods is that mined features are problem dependent, causing poor generalization ability across different applications. Recent studies show that we can represent networked data (e.g., citation networks and social networks) as low-dimensional vectors with rich structure and content information preserved, which can then greatly facilitate many downstream tasks such as classification and clustering. In this article, we focus on the problem of Web service network embedding, which aims to learn low-dimensional vectors to represent services by encoding both Mashup-API composition structure and service functional content. We first propose a novel probabilistic topic model to predict potential links between Mashups and APIs in the service network. Then, we develop a Service Graph Convolutional Network (Service-GCN) to learn vector representations of services, where each service (e.g., Mashup or API) forms its representation through message passing between neighborhood services over the network. We evaluate the network embedding quality on two real-world datasets for downstream classification and clustering tasks. Experimental results show that the average performance of our method improves 20.7 percent (Micro-F1) in service classification and 19.0 percent (Accuracy) in Mashup clustering compared to the state-of-the-art, which verified the effectiveness of the proposed approach for learning vector representations of Web services.

Index Terms—Software services, Web services, representation learning, network embedding, topic model

1 Introduction

The appearance of service-oriented architectures (SOAs) has greatly changed the fashion of developing software systems from monolithic, static and centralized structures to modular, dynamic and distributed ones [1]. The system (a.k.a. Mashup) as a whole is generally designed and implemented by composing a collection of interactive Web-based software services (e.g., API services), where each service provides access to a well-defined set of functionalities. On the other hand, the accumulation of a broad range of Web services on the Internet has posed critical challenges on many real-world problems such as service classification or clustering [2], [3], service discovery [4], [5], service composition [6], [7] and service annotation [8], [9]. Overcoming these problems would substantially ease the development process of distributed software applications. For example, Web service classification according to the functionalities would help users better grasp functional structures of the entire service repository [2]. Vertical service retrieval within a specific functional domain would dramatically reduce the searching space and improve the matching precision [3].

However, a critical challenge of these analytic tasks and applications is to seek effective service representations (e.g., low-dimensional vectors) by which functional similarities between services can be accurately calculated. Traditional methods mainly explore service description content and learn a topic distribution vector to represent each Web service [2], [10]. For example, to obtain Web service representations for the clustering task, Shi *et al.* [2] proposed a word embedding augmented topic model that leverages friend words to refine the topic learning. Similarly, to support quality Web service recommendation, Cao *et al.* [3] proposed a two-layer topic learning model to derive representations of Mashups from their functional descriptions. Yet, one common issue for existing methods is that the carefully derived representations are typically task dependent with encoded information only benefiting the given task [3], [11]. In addition, service representations learned using topic models may not be fine-grained or discriminative to measure the closeness between services, i.e., the topic distribution vector presents a set of discrete probability values that are not efficient to reveal the distance or similarity between services.

Beyond the content information, services normally have frequent connections with each other forming a complex service linking network [2], [4], where nodes can be Mashup

- *Min Shi and Jianxun Liu are with the Hunan Provincial Key Laboratory for Services Computing and Novel Software Technology, Hunan University of Science and Technology, Xiangtan 411201, China. E-mail: {toshimin132, ljx529}@gmail.com.*
- *Yuan Zhuang is with the State Key Laboratory of Information Engineering in Surveying, Mapping and Remote Sensing, Wuhan University, Wuhan 430079, China. E-mail: yuan.zhuang@whu.edu.cn.*
- *Yufei Tang and Xingquan Zhu are with the Department of Electrical Engineering and Computer Science, Florida Atlantic University, Boca Raton, FL 33431 USA. E-mail: {tangy, xzhu3}@fau.edu.*
- *Maohua Lin is with the Department of Ocean and Mechanical Engineering, Florida Atlantic University, Boca Raton, FL 33431 USA. E-mail: mlin2014@fau.edu.*

Manuscript received 29 November 2020; revised 20 July 2021; accepted 4 August 2021. Date of publication 12 August 2021; date of current version 9 December 2022.
(Corresponding author: Jianxun Liu.)
Digital Object Identifier no. 10.1109/TSC.2021.3103481

1939-1374 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:09:54 UTC from IEEE Xplore. Restrictions apply.

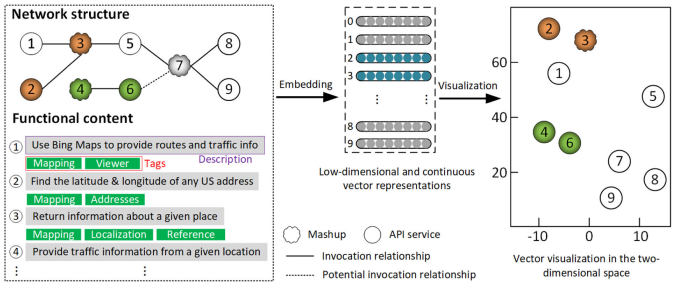


Fig. 1. The learning target. We aim to represent services with low-dimensional vectors in which the service network structure and functional content are well preserved. After that, services with similar network structure and functional content are closed in the learned latent space.

and API services and edges indicate composition relationships between them. Network structures inherently reveal rich explicit and implicit semantic relevance (e.g., similar functional properties) between services through edge connections [5], [9], i.e., two Mashups linking to an identical API service could possess similar functional properties. Recent studies on network science show that we can learn vector representations of nodes from the complex network structures [12], [13], which assumes that nodes with similar topology structure have similar representations. In this paper, we focus on Web service network embedding, which aims to represent each service (e.g., Mashup and API) by a low-dimensional vector with preserved functional similarities between services. The study is based on three observations: 1) A Mashup and an API with a direct link (e.g., composition relationship) in the network have similar functionalities (e.g., semantically related); 2) Two Mashups with links to identical API services in the network have similar functionalities; 3) Two services should have similar vector representations if they have similar functionalities.

In addition, a plethora of studies show that a complex network could suffer from many missing links [14], [15]. For example, in a social network two users do not have a link not because they are not friend, but because they might be unaware of each other's existence. Analogically, a Mashup does not compose an API service not meaning they are functionally dissimilar but because other similar API services have been selected. Since we aim to map services with similar functionalities in similar vector representations, exploring potential/missing composition relations would enhance affinities between Mashup and API services in the network for enriched representation learning.

Taken together, to learn informative and discriminative service representations, we argue that it is necessary to simultaneously encode observed network links, potential links and description contents of services. Fig. 1 shows the learning target. We integrate network structure (e.g., both observed and potential links) and content information (e.g., descriptions and tags) to learn low-dimensional representations of services, such that services with similar topological structure (e.g., node pairs $< 2, 3 >$ and $< 4, 6 >$) can be mapped to be similar in the embedding space (e.g., close by in the two-dimensional euclidean space). To reach this target, we first propose a novel probabilistic topic model to predict potential link relationships between Mashup and API services. Then, we adopt an efficient convolutional learning model to learn service representations with preserved network structure and content information.

In conclusion, our contribution is threefold:

1. We propose an attention-based relational topic model, Att-RTM, to learn latent topics from Web service text contents and composition relationships, which is then used to predict potential links between Web services. An attention mechanism is introduced that leverages tags as prior information to highlight functional-related words in the text content for effective topic learning and composition link prediction.
2. We develope Service-GCN based on the graph convolutional networks to combine network structure and content information for Web service representation learning. During convolutional learning, each service forms its representation by aggregating features from all linked neighborhood services.
3. We demonstrate the advantage of learning service representations through three popular tasks on two real-word datasets. Experimental results verified the effectiveness of our proposed approach.

The rest of this paper is organized as follows. Section 2 surveys related work. Section 3 formulates the problem of Web service network embedding. Section 4 presents the link prediction and convolutional representation learning models. Section 5 describes experimental evaluation. Section 6 presents a discussion of the studied problem and method. Finally, Section 7 concludes the paper.

2 Related Work

In this section, we review existing work relevant to the studied problem, including Web service representation learning and network representation learning in general.

2.1 Web Service Representation Learning

Web service representation learning often appears as a core component of many fundamental tasks such as Web service clustering [2], [4], annotation [9], [16], and recommendation in the software development [17], [18], where services need to be represented as numerical vectors to enable similarity measurement and calculation.

Platzer *et al.* [19] utilized a Vector Space Model (VSM) to represent the service description with a high-dimensional vector, with each keyword constituting a relevant dimension. An obvious drawback of VSM is that the vector dimensionality is subject to the vocabulary size, which may suffer from the problems of dimensionality curse and feature sparsity. Ma *et al.* [10] proposed to use the Singular Value Decomposition (SVD) to represent services in the latent semantic space. It can significantly reduce the feature vector dimensionality and meanwhile capture the concept-level similarities between Web services. However, SVD involves expensive computation complexity especially when the input feature matrix is very large. To address this issue, researchers shifted their focus to the latent topic-based feature representations of Web services [2], [3], [20], [21]. For example, Cassar *et al.* [22] proposed to use Probabilistic Latent Semantic Analysis (PLSA) and Latent Dirichlet Allocation (LDA) to elicit semantics from descriptions of

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:09:54 UTC from IEEE Xplore. Restrictions apply.

services and then represent each service with a low-dimensional topic vector. However, the naïve PLSA and LDA models work well only when a large volume of training data are provided [2], [23]. It is problematic to learn quality representations of services due to realities of feature sparsity (e.g., short text) and noise (e.g., irrelevant words) [3], [24]. To alleviate these issues, Li *et al.* [25] and Cao *et al.* [26] proposed to use link relationships between services for improved description semantic extraction and representation learning, where linked services are assumed to have similar topic distributions. Shi *et al.* [2] proposed a word embeddings augmented LDA model that leverages friend words to refine the topic assignment and semantic representation for each current sampling word.

However, above methods are typically content-centered despite that few works have leveraged service composition links as auxiliary side information. The obvious drawback is that they fail to explicitly model the rich structure relationships between services [27]. Recently, some works turn to focus on network-based service representation learning in that services with similar topology in the network are represented in similar vectors. For example, Adeleye *et al.* [28] adopted a random walk-based embedding method to embed Web service networks, where node relationships are captured by truncated random walk over the network. Our previous work [29] proposed a translation-based method, where the representation of each Mashup service is forced to resemble the combined representation of all member API services. However, the method only models the direct link relationships between Web services. As discussed previously, potential links can be mined to boost relationships between services in the network. Existing works often focus on the unsupervised service representation learning. In fact, Web services of varying functions normally come from different functional domains/categories and such domain information can be viewed as labels to design more effective representation learning process in an supervised manner.

In comparison, we propose a novel approach to embed Mashup and API services by simultaneously encoding observed network structure, potential network structure and content information. In addition, our model is trained by end-to-end supervised classification and representation learning, which is a stark difference from existing methods.

2.2 General Network Representation Learning

In general, existing network representation learning methods can be classified into two groups: structure-preserving [12], [30] and attribute-preserving methods [26], [31].

Structure-preserving methods encode only the network structure information such as neighborhood relationships between nodes. DeepWalk [30] first performs a truncated random walk over the whole network to capture the neighborhood relationships. Then, it adopts an efficient neural model called Skip-Gram [32] to derive representations of all nodes in the network. Node2vec [12] adopts a more flexible way than DeepWalk to capture the diversity of connectivity patterns in a network. It designs a second-order random walk strategy to sample neighborhood nodes, which can embed nodes belonging to the same network community closely. Similarly, LINE [33] is proposed for large-scale network embedding that can simultaneously encode first-order (e.g., direct neighbors) and second-order (e.g., 2-hop neighbors) node affinities. However, above methods neglect the rich node content information which capture node similarities from the non-structure perspective.

Attribute-preserving methods seek to encode both network structure and content information. Tu *et al.* [34] proposed a semi-supervised network embedding method, MMDW, by leveraging the labeling information of nodes. It performs representation learning based on the DeepWalk-derived matrix factorization. Chen *et al.* [35] and Huang *et al.* [31] also proposed to incorporate node labels for representation learning where nodes with identical labels are mapped to be close in the embedding space. Recent studies show that text contents are helpful auxiliary information to enhance the structure-based representation learning, especially when the network structure is sparse. For example, Le *et al.* [36] proposed a PLANE model that combines both link relationship and content information for large-scale citation network representation learning. However, only observed direct links are considered in PLANE and the accuracy improvement overly depends on the network connection density. Yang *et al.* [37] proposed a matrix factorization-based method that leverages rich text content to refine the network structure, but the computation is very expensive. Liao *et al.* [26] proposed an attribute-assisted method to embed social networks by preserving both structure- (e.g., global net-work structure) and attribute- (e.g., gender, location and text description) based proximities.

3 Problem Definition & Preliminaries

This section defines the problem of **W**eb **S**ervice **N**etwork **E**mbedding (WSNE), followed by the introduction of some preliminaries. The terms *network embedding* and *network representation learning* are used interchangeably.

3.1 Problem Definition

Definition 1 (Web Services). *Web services indicate API services or Mashup services. Each API service a is associated with a text content represented by $doc_a = \{w_{a,1}, w_{a,2}, \ldots, w_{a,N_a}\}$, and a tag set represented by $tag_a = \{t_{a,1}, t_{a,2}, \ldots, t_{a,S_a}\}$. N_a and S_a are number of words in the text content and size of the tag set, respectively. The same for each Mashup service with $doc_m = \{w_{m,1}, w_{m,2}, \ldots, w_{m,N_m}\}$ and $tag_m = \{t_{m,1}, t_{m,2}, \ldots, t_{m,S_m}\}$, where N_m and S_m are number of words in the text content and size of the tag set.*

Definition 2 (Composition Relationships). *A Mashup service m comprises multiple member API services represented by $CR(m) = \{a_1, a_2, \ldots, a_{|CR(m)|}\}$, where $|CR(m)|$ indicates the number of API services that Mashup m invokes. The composition relationship represents that a service a_i is a member of the Mashup service m.*

Definition 3 (Web Service Network). *A Web service network is represented as $G = (\mathbf{V}, \mathbf{E}, \mathbf{C})$, where $\mathbf{V} = \{v_i\}_{i=1,\ldots,|\mathbf{V}|}$ is a set of nodes consisting of all API and Mashup services, $\mathbf{E} = \{e_{i,j}\}_{i=1,\ldots,A;j=1,\ldots,M}$ is a set of edges linking these nodes, e.g., if API service a_i is a member of Mashup service m_j, then $e_{i,j} = 1$, or $e_{i,j} = 0$ otherwise. $\mathbf{C} = \{doc_i\}_{i=1,\ldots,|\mathbf{V}|}$ denotes the textual content describing the corresponding nodes. A and M are the*

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:09:54 UTC from IEEE Xplore. Restrictions apply.

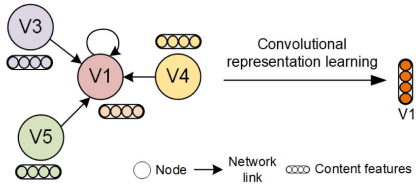


Fig. 2. The convolutional learning mechanism of GCN.

numbers of APIs and Mashups in the Web service repository, respectively. In addition, each service node v_i belongs to a major functional domain $c_i \in \mathbf{L}$, where $\mathbf{L}$ denotes the set of functional domains. Services from an identical functional domain present similar functional properties.

Formulation of WSNE. Given the network G, the task of WSNE aims to represent each Mashup or API node $v_i \in \mathbf{V}$ with a continuous low-dimensional vector $\mathbf{h}_{v_i} \in \mathbb{R}^d$, i.e., learning a mapping $f : G \rightarrow \{\mathbf{h}_{v_i}\}_{i=1,\ldots,|\mathbf{V}|}$ so that observed network structure, potential network structure and content information can be well preserved. We focus on the supervised network embedding where the learned vector representation of each service node v_i is trained to predict its corresponding functional domain/label c_i.

Through the Web service embedding learning, we aim to answer the following research questions: 1) Is network structure-centered representation learning effective to capture similarities between Web services? 2) Is the proposed model effective for Web service representation learning? 3) Is the potential link prediction between API and Mashup services beneficial for the Web service network embedding?

3.2 Preliminaries

WSNE contains two sequential components: 1) predicting the potential links between Mashups and APIs; and 2) learning the vector representations of Web services. In this paper, the proposed link prediction model extends from the Relational Topic Model (RTM) [27] and the convolutional service representation learning builds on the Graph Convolutional Network (GCN) [38]. This section briefly describes the learning mechanisms of RTM and GCN.

3.2.1 Relational Topic Model

RTM is a model of documents and the links between them. For each pair of documents, RTM models their link as a binary random variable conditioned on their text contents. Therefore, RTM can be seen as a predictive model capable of predicting links between nodes in a network based on node contents, i.e., hyperlink prediction between Web pages based on the similarity of Web page contents.

The parameters to RTM are T distributions $\varphi_{1:T}$ over terms, a T-dimensional topic distribution θ_d for each document d, and a function ψ that provides binary probabilities. RTM assumes that a set of observed documents D and binary links $y_{1:D,1:D}$ between them are generated by a procedure:

1) For each document $d \in D$:
   a) Draw a topic proportion $\theta_d|\alpha \sim \mathrm{Dir}(\alpha)$.
   b) For each word $w_{d,n} \in d$:
      i) Draw a topic assignment $z_{d,n}|\theta_d \sim \mathrm{Mult}(\theta_d)$;
      ii) Draw a word $w_{d,n}|z_{d,n}, \varphi_{1:T} \sim \mathrm{Mult}(\varphi_{z_{d,n}})$.
2) For each pair of documents $d, d' \in D$:
   a) Draw a binary link $y_{d,d}|z_d, z_{d'} \sim \psi(\cdot|z_d, z_{d'})$.

where $w_{d,n}$ represents the nth word in d, Dir and Mult represent the Dirichlet and multinomial distributions respectively. ψ is the *link probability function* that defines a distribution over the links between documents conditioned on their topic assignments, i.e., the presence of a link between d and d' depends on the topic assignments z_d and $z_{d'}$. Following the generative steps in 1) and 2), the training of RTM aims to estimate posterior distributions $\varphi_{1:T}$, $\theta_{1:D}$ and ψ based on all observed (e.g., prior distributions) words and links in D.

3.2.2 Graph Convolutional Network

GCN extends from the traditional convolutional neural network to directly learn on the graph-structured data. Given an input network $G = (\mathbf{V}, \mathbf{E}, \mathbf{X})$, where $\mathbf{V}$ and $\mathbf{E}$ are sets of nodes and edges. $\mathbf{X} \in \mathbb{R}^{|\mathbf{V}|\times h}$ (h is the number of features) is the feature matrix (e.g., text contents of Web services) of all $|\mathbf{V}|$ nodes, i.e., $\mathbf{X}_j \in \mathbb{R}^h$ is node v_j's feature vector. GCN learns a vector representation $\mathbf{h}_{v_i} \in \mathbb{R}^d$ for each node $v_i \in \mathbf{V}$ by preserving its network structure and content information. GCN adopts a convolutional learning process where each node v_i aggregates information from all linked neighbors $\mathbf{N}(v_i)$, i.e., V1 generates its vector representation by aggregating (e.g., mean-pooling) features from neighbors (e.g., include itself) V3, V4, V5 and V1 shown in Fig. 2. This convolutional learning process can be represented as

$$\mathbf{h}_{v_i} = f\left(\sum_{v_j \in \mathbf{N}(v_i)} \mathbf{h}_{v_j}\mathbf{W} + \mathbf{b}\right), \forall v_i \in \mathbf{V}, \tag{1}$$

where $\mathbf{W} \in \mathbb{R}^{h\times d}$ is the learnable model parameter and $\mathbf{b}$ is the bias. f is an activation function, i.e., $ReLu$ represented by $f(x) = max(0, x)$. $\mathbf{h}_{v_j} \in \mathbb{R}^h$ is the initial feature vector, i.e., $\mathbf{h}_{v_j} = \mathbf{X}_j$, and $\mathbf{h}_{v_i} \in \mathbb{R}^d$ is node v_i's vector representation output by GCN. For a single-layer GCN, the vector representations $\mathbf{X}^{(1)} \in \mathbb{R}^{|\mathbf{V}|\times d}$ for all nodes based on Eq. (1) can be written as

$$\mathbf{X}^{(1)} = f\left(\widetilde{\mathbf{A}}\mathbf{X}\mathbf{W}\right), \tag{2}$$

where $\widetilde{\mathbf{A}} = \mathbf{D}^{-\frac{1}{2}}(\mathbf{A} + \mathbf{I})\mathbf{D}^{-\frac{1}{2}}$ is the normalized symmetric adjacency matrix, $\mathbf{A}$ is the adjacency matrix of network G, $\mathbf{I}$ is the identity matrix and $\mathbf{D}_{ii} = \sum_j \mathbf{A}_{ij}$ is the degree matrix. A single-layer GCN only captures first-order neighborhood relations for representation learning. To preserve high-order relations, more GCN layers can be stacked and new representations for all nodes at the kth layer are calculated by

$$\mathbf{X}^{(k+1)} = f\left(\widetilde{\mathbf{A}}\mathbf{X}^{(k)}\mathbf{W}^{(k)}\right), \tag{3}$$

where $\mathbf{X}^{(k)}$ is the feature input at the kth layer which is also the output from the $(k-1)^{th}$ layer, and $\mathbf{W}^k$ denotes the learnable model parameter at the kth GCN layer.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:09:54 UTC from IEEE Xplore. Restrictions apply.

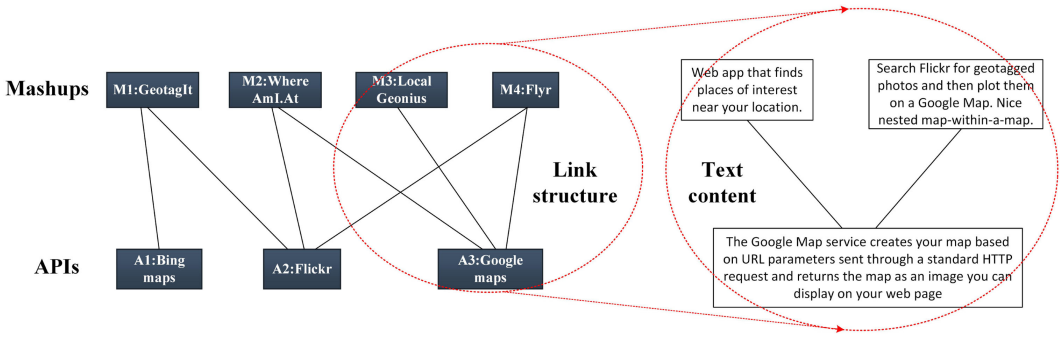


Fig. 3. The Web service network containing Mashup and API services together with their text contents.

4 Methodology Overview

The proposed approach comprises two components including Web service link prediction and Web service network embedding learning. They are described as follows.

4.1 Web Service Link Prediction

This study aims to map functionally similar Web services in similar vector representations. The input is a Web service network containing both link structure and text content of services shown in Fig. 3. As discussed previously, the potential links between Mashups and APIs can be beneficial for enhanced representation learning. For example, when a Mashup and an API have similar text content but without a direct link in the network, the content similarity cannot be efficiently captured because the adopted convolutional representation learning relies on the feature passing through edge links shown in Fig. 2. To alleviate this issue, we propose a novel topic model to predict the missing of potential links in the service network, which can predict links between Mashups and APIs conditioned on their text contents.

The proposed topic model extends from RTM (refer to Section 3.2.1). The basic RTM learns the topic distribution of a document by treating all constituent words equally. However, Web service text contents are functional-oriented, meaning that some words are functional-related while others are less relevant. This observation inspired us to highlight functional-related words while inferring the latent topics of services. We quantify the significance of each word in a service text content according to its closeness (e.g., semantic similarity) to the corresponding service functional tags. To this end, we map all words and tags into a unified semantic space using Word2vec [32] in which the semantic similarity between words and tags can be computed. Word2vec is a shallow neural network model to represent words with low-dimensional vectors capable of capturing the semantic relevance between words, i.e., words known to be synonyms are closed in the learned space, whereas antonyms tend to have larger distance. Assume we have obtained semantic vectors of words and tags based on Word2vec, the significance or attention weight $p_{a,i}$ of a word $w_{a,i}$ (or $w_{m,i}$) can be computed by

$$p_{a,i} = cos(WE(w_{a,i}), FC(tag_a)), \tag{4}$$

where

$$FC(tag_a) = \frac{1}{S_a}\sum_{j=1}^{S_a} WE(t_{a,j}), \tag{5}$$

where $WE(w_{a,i})$ and $WE(t_{a,j})$ represent the semantic vectors of word $w_{a,i}$ and tag $t_{a,j}$, respectively. $FC(tag_a)$ represents the average semantic of all S_a functional tags attached to service a. The cosine similarity is adopted in Eq. (4).

By attending different words differently, we propose Attention-based RTM (Att-RTM) for Web service link prediction shown in Fig. 4. Notations used in Att-RTM are summarized in Table 1. Simialr to RTM (refer to Section 3.2.1), parameters for Att-RTM are T distributions $\varphi_{1:T}$ over terms, T-dimensional topic distributions $\boldsymbol{\theta}_a$ or $\boldsymbol{\theta}_m$ for each API or Mashup service, and a link probability function ψ which defines a distribution over the links between API and Mashup services. Att-RTM assumes that API and Mashup service text contents and their links are generated by a process summarized in Algorithm 1. The training takes all service text contents and respective word attention weights (e.g., Eq. (4)) as inputs. First, the model generates a distribution of words for each topic t (lines 1-3). Then, it generates the topic distribution of each API (line 5) and each Mashup (line 13), and meanwhile generates all its constituent words with different attention weights for each API (lines 6-10) and each Mashup (lines 14-18). Finally, Att-RTM generates a binary indicator for each pair of API and Mashup services (lines 20-24), which depends on the topic assignments $\mathbf{z}_a = [z_{a,1}, z_{a,2}, \ldots, z_{a,N_a}]$ and $\mathbf{z}_m = [z_{m,1}, z_{m,2}, \ldots, z_{m,N_m}]$ generating their text contents. Similar to literature [27], ψ can be the link probability function defined as

$$\psi(y_{m,a} = 1) = \exp(\eta^T(\bar{\mathbf{z}}_a \circ \bar{\mathbf{z}}_m) + v), \tag{6}$$

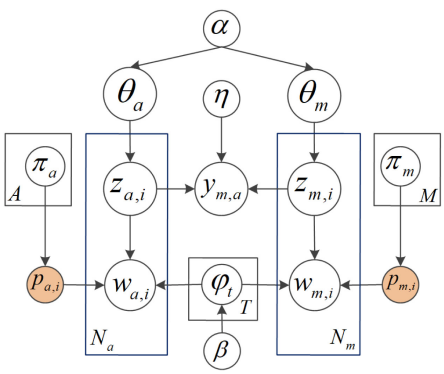


Fig. 4. Plate notation of the proposed Att-RTM.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:09:54 UTC from IEEE Xplore. Restrictions apply.

TABLE 1
Notations Used in the Att-RTM Model

Symbols	Description
T	The number of topics
φ_t	Word distribution for topic $z_{a,i}$ or $z_{m,i}$
$\boldsymbol{\theta}_a$	Topic distribution of API service a
$\boldsymbol{\theta}_m$	Topic distribution of Mashup service m
ψ	The linkage probability function
A	The number of API services
M	The number of Mashup services
$\lvert\mathbf{V}\rvert$	The total number of API and Mashup services
$\mathcal{W}$	The word set for all Web service descriptions
N	The number of words
N_a	The number of words contained in API service a
N_m	The number of words contained in Mashup service m
$w_{a,i}$	The ith word in API service a
$w_{m,i}$	The ith word in Mashup service m
$z_{a,i}$	The topic assignment for word $w_{a,i}$
$z_{m,i}$	The topic assignment for word $w_{m,i}$
$y_{m,a}$	A binary indicator between API a and Mashup m, i.e., $y_{m,a} = 1$ means there is a link between a and m
v	The intercept value used in ψ
π_a	A vector that records the weight of each word in API service a
π_m	A vector that records the weight of each word in Mashup service m
$p_{a,i}$	The weight of the ith word in API service a
$p_{m,i}$	The weight of the ith word in Mashup service m
α, β, η	Hyper-parameters in the model

where $\bar{\mathbf{z}}_a = \frac{1}{N_a}[n_{a,1}, n_{a,2}, \ldots, n_{a,T}]$ and $\bar{\mathbf{z}}_m = \frac{1}{N_m}[n_{m,1}, n_{m,2}, \ldots, n_{m,T}]$ are statistic representations of the topic proportions of API a and Mashup m, respectively. $n_{a,i}$ and $n_{m,i}$ denote the numbers of words in API a and Mashup m belonging to topic i. $\circ$ denotes the Hadamard product capturing the similarity between topic distributions of API a and Mashup m, parametered by coefficient η and intercept v. Eq. (6) allows the topic assignments that are used to generate service text contents to in return predict the link probability between API and Mashup services.

The training of Att-RTM aims to maximize the following log-likelihood over all observed words $\mathcal{W}$ in $doc_{1:A+M}$ and composition links $\mathbf{Y}$ between API and Mashup services

$$\mathcal{L} = \sum_{d=1}^{|\mathbf{V}|} \sum_{i=1}^{N_d} log \sum_{t=1}^{T} P(w_{d,i}|\varphi_{z_{d,i}}) p(z_{d,i}|\theta_i) + \sum_{(a,m)} logP(y_{m,a}|z_a, z_m, \eta, v), \tag{7}$$

where d can be either a or m. The posterior parameters $\theta_{1:A+M}$ in Eq. (7) for link prediction can be approximated by the Markov Chain Monte Carlo (MCMC) method [39]. During the training, a Markov chain is established that iteratively updates the topic assignments of words. The topic update rule for word $w_{a,i}$ (similar update rule applies to word $w_{m,i}$) is given by

$$P(z_{a,i} = t|\mathbf{z}_{\neg(a,i)}, \mathcal{W}, \mathbf{Y}) \propto \prod_{m_{1:M}, y_{a,m}=1} exp\left(\frac{\eta}{|N_a|}\right) \bar{z}_{m,t} \cdot P(z_{a,i} = t|\mathbf{z}_{\neg(a,i)}, \mathcal{W}), \tag{8}$$

where

$$P(z_{a,i} = t|\mathbf{z}_{\neg(a,i)}, \mathcal{W}) = \frac{\sum_{h=1}^{A+M} \sum_{j=1}^{N_h} DF(w_{h,j} = w_{a,i} \& z_{h,j} = t) + \beta}{\sum_{h=1}^{A+M} \sum_{j=1}^{N_h} DF(z_{h,j} = t) + N\beta} \cdot \frac{\sum_{j=1}^{N_a} DF(z_{a,j} = t) \cdot_{a,j} + \alpha}{\sum_{j=1}^{N_a} p_{a,j} + T\alpha}, \tag{9}$$

Algorithm 1. The Generative Algorithm of Att-RTM

Input: API and Mashup text contents $doc_{1:A+M}$; word attention weights $\pi_{1:A+M}$ in text contents
Output: Latent topic distributions $\theta_{1:A+M}$; word distributions of topics $\varphi_{1:T}$; link distributions ψ between API and Mashup services

for $t = 1 : T$ **do**
  Draw a distribution over all words $\beth_t|\beta \sim Dirichlet(\beta)$ based on Eq. (9).
end
for $doc_a, a = 1 : A$ **do**
  Draw a distribution over all topics $\theta_a|\alpha \sim Dirichlet(\alpha)$ based on Eq. (10).
  for $w_{a,i}, i = 1 : N_a$ **do**
    Conditional on θ_a choose a topic $z_{a,i}|\theta_i \sim Multinomial(\theta_a)$ based on Eq. (8);
    Conditional on $z_{a,i}$ choose a word $w_{a,i}|z_{a,i}, \varphi_{1:T} \sim Multinomial(\varphi_{z_{a,i}})$;
    Assign an attention weight $p_{a,i}$ from π_a for word $w_{a,i}$ based on Eq. (4).
  end
end
for $doc_m, m = 1 : M$ **do**
  Draw a distribution over all topics $\theta_m|\alpha \sim Dirichlet(\alpha)$ based on Eq. (10).
  for $w_{m,i}, i = 1 : N_m$ **do**
    Conditional on θ_m choose a topic $z_{m,i}|\theta_i \sim Multinomial(\theta_m)$ based on Eq. (8);
    Conditional on $z_{m,i}$ choose a word $w_{m,i}|z_{m,i}, \varphi_{1:T} \sim Multinomial(\varphi_{z_{m,i}})$;
    Assign an attention weight $p_{m,i}$ from π_m for word $w_{m,i}$ based on Eq. (4).
  end
end
for $doc_a, a = 1 : A$ **do**
  for $doc_m, m = 1 : M$ **do**
    Draw a binary link indicator $y|\mathbf{z}_a, \mathbf{z}_m \sim \psi(\cdot|\mathbf{z}_a, \mathbf{z}_m)$ based on Eq. (6)
  end
end

where $\mathbf{z}_{\neg(a,i)}$ denotes the vector of word topic assignments excluding $z_{a,i}$. DF is a decision function subject to the given condition, i.e., if the condition is true then return 1, or 0 otherwise. For example, $DF(w_{h,j} = w_{a,j} \& z_{h,j} = t)$ means word $w_{h,j}$ is identical to word $w_{a,i}$ and meanwhile word $w_{h,j}$

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:09:54 UTC from IEEE Xplore. Restrictions apply.

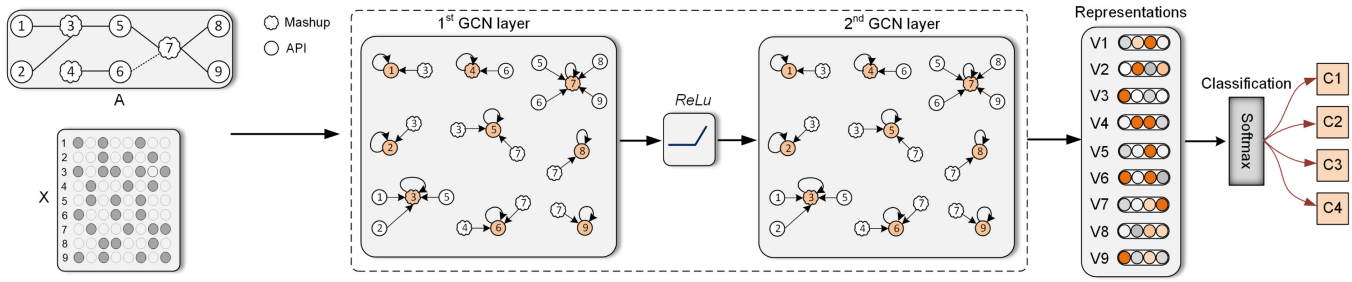


Fig. 5. The two-layer Service-GCN model for Web service network embedding. The adjacency matrix $\mathbf{A} \in \mathbb{R}^{|\mathbf{V}|\times|\mathbf{V}|}$ represents network structures involving both observed and predicted links between API and Mashup services. $\mathbf{X} \in \mathbb{R}^{|\mathbf{V}|\times N}$ represents text contents for all services. Embeddings are trained by a supervised classification of service functional domains.

belongs to topic t. If DF returns 0, it means the corresponding index h does not count in the calculation. Eq. (8) has considered both observed service links and contents for topic sampling. Therefore, the derived latent topics can explain both service contents and composition links, i.e., APIs and Mashups that have similar text contents are more likely to have a composition link and vice versa. In addition, instead of treating different word features equally, we use attention weights to differentiate word importances (e.g., $p_{d,j}$ in Eq. (4)), thereby functional-related words will contribute more while inferring latent topics of services.

After the training, each word $w_{d,i}$ (e.g., $w_{a,i}$ or $w_{m,i}$) in the text content doc_d (d can be a or m) will be assigned with a latent topic $z_{d,i}$. Then, the distribution of service d at topic t can be computed by

$$\theta_d = \frac{\sum_{j=1}^{N_d} DF(z_{d,j} = t) \cdot p_{d,j} + \alpha}{\sum_{j=1}^{N_d} p_{d,j} + T\alpha}. \tag{10}$$

Finally, the likelihood to predict a link between API a and Mashup m based on their text contents is calculated by

$$P(doc_a, doc_m) = exp(\eta^T(\theta_a \circ \theta_m) + v). \tag{11}$$

In this paper, we predict top-K most likely missing API service links for each Mashup service according to likelihoods by Eq. (11). In this experiment, we set K as 1 because it delivers the best embedding learning performance (e.g., refer to Fig. 9b for the impact of K).

4.2 Web Service Network Embedding

This section details the Web service network embedding by simultaneously encoding observed service links, predicted service links (refer to Section 4.1) and service text contents shown in Fig. 3 through a convolutional learning based on GCN (refer to Section 3.2.2).

We develop Service-GCN shown in Fig. 5 to obtain representations of Mashup and API services, which adopts a convolutional learning mechanism (refer to illustration in Fig. 2) with supervised training. At the first convolution layer, both Mashup and API nodes perform feature aggregation to form their respective representations as

$$\mathbf{X}^{(1)} = f\left(\widetilde{\mathbf{A}}\mathbf{X}\mathbf{W}_0\right), \tag{12}$$

where $\widetilde{\mathbf{A}} = \widetilde{\mathbf{D}}^{-\frac{1}{2}}(\mathbf{A}+\mathbf{I})\widetilde{\mathbf{D}}^{-\frac{1}{2}}$ is the normalized symmetric adjacency matrix, and $\mathbf{A} \in \mathbb{R}^{|\mathbf{V}|\times|\mathbf{V}|}$ is the adjacency matrix of Web service network. $\mathbf{X} \in \mathbb{R}^{|\mathbf{V}|\times N}$ represents the input content feature matrix of all $|\mathbf{V}|$ Web services, i.e., each service corresponds to a row representing its text content. N is the total number of unique words. $\mathbf{W}_0 \in \mathbb{R}^{N\times r}$ denotes learnable parameters at the first GCN layer, where r is the dimension of output embedding vector.

The embedding vectors output by the first layer can be further fed into the second layer for recursive convolutional learning, which allows second-order neighborhood relations between Web services to be captured. For instance, since representation of node 5 in Fig. 5 has encoded features from node 3 after the first-layer convolution learning, when node 7 aggregates features from node 5 again at the second-layer convolution learning, node 3's features will be incorporated in node 7's new embedding vector. At the second convolutional layer, embedding vectors of Mashup and API services are calculated as (refer to Section 3.2.2 for explanation)

$$O = f(\widetilde{\mathbf{A}}\mathbf{X}^{(1)}\mathbf{W}_1), \tag{13}$$

where $\mathbf{W}_1 \in \mathbb{R}^{r\times d}$ denotes learned parameters.

To train model parameters $\mathbf{W}_0$ and $\mathbf{W}_1$, embedding dimension of the final layer is set to be the number of Web service functional domains, and embeddings are subsequently passed through a *softmax* classifier to perform multi-class classification training by

$$\mathcal{Z} = softmax(O) = \frac{exp(O)}{\sum_i exp(O_i)} \tag{14}$$

$$\mathcal{L}_{gcn} = -\sum_{v_i \in \mathbf{V}_l} \sum_{j=1}^{d} \mathcal{Y}_{ij} \ln \mathcal{Z}_{ij}. \tag{15}$$

Eq. (14) transforms the embedding vector of each Web service into a probability vector where each dimension indicates the likelihood of the service being classified to a corresponding functional domain. After the transformation, each Web service will be assigned with a most likely functional domain. Eq. (15) represents the cross-entropy loss of classification results over the labeled (e.g., with functional domains known) Web services $\mathbf{V}_l$, where $\mathcal{Y} \in \mathbb{R}^{|\mathbf{V}|\times d}$ is the one-hot domain indicator matrix of all Web services, i.e., each service corresponds to a vector with only one entry equals to one, while others are zeros. Finally, the learning tries to minimize the classification loss using the stochastic gradient decent (SGD) algorithm through the back-propagation process [40].

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:09:54 UTC from IEEE Xplore. Restrictions apply.

5 Performance Evaluation

We validate the quality of learned embedding vectors by conducting downstream tasks on two real-word datasets, including supervised node classification, unsupervised node clustering and node embedding vector visualization.

5.1 Datasets

Webservice. This dataset has been widely used in previous research such as service recommendation [3], [25] and clustering [2], [22]. It contains 19,718 Web services, where the numbers of API and Mashup services are 13,460 and 6,258, respectively. There are 26,246 composition links between API and Mashup services. Each service is associated with a functional description and several functional tags. The average numbers of words and tags per service are 36 and 4, respectively. Each service corresponds to a major functional domain (e.g., "Video" and "Music") out of 478 domains. The composition linking rate is $\frac{26246}{13460\times6258} = 0.031\%$, which is very low, partially because some functionally relevant API and Mashup services are not connected on the network.

We build the Web service network based on observed and predicted link relationships between Mashup and API services. Since nodes rely on their neighbors for convolution representation learning in this paper, isolated services that have no links with others in the network are removed. After the link prediction (e.g., the number of predicted links $K = 1$), we learn service representations based on a network involving 10,234 Web services and 38,764 links.

Wiki. We also use a Web page linking network which can be analogical to a Web service network, i.e., both Web page and Web service are presented as text contents and links imply the respective nodes share similar text contents. Wiki contains 2,405 Web pages and 17,981 links between them. Similar to the Web service, each Web page is presented as a text description with 647 words in average. Each Web page belongs to a relevant category out of 17 categories, which can be used as labels for supervised training in the experiment. Since Web pages do not have tag information, Att-RTM cannot be used as the link prediction model. Instead, we use RTM to predict potential links between Web pages. Finally, all predicted links are combined with the original links to form a single Web page network.

5.2 Comparative Methods

We compare the following baseline methods:

- *DeepWalk* [30] This method preserves only network structure information based on the truncated random walk over the whole network. It learns node representations using the Skip-Gram model.
- *Node2vec* [12] Compared with DeepWalk, this method adopts a more flexible random walk process by simultaneously capturing the local structures and global structures of the network. Node2vec preserves only the network structure information.
- *LINE* [33] It is a structure-preserving method, which encodes both local and global network structures. This method adopts an edge-sampling algorithm to address the limitation of the classical stochastic gradient descent and improve both the effectiveness and the efficiency of the inference.
- *RTM* [27] It is a relational topic model that utilizes both node content and structure information for latent topics learning. It directly uses the learned latent topic distributions to represent nodes.
- *Att-RTM* It is the proposed Att-RTM model for link prediction in Fig. 4. Simial to RTM, Att-RTM learns latent topic distributions to represent services. Compared with RTM, the only difference of Att-RTM is to introduce an attention mechanism to highlight functional words when inferring the latent topics of Web services.
- *PLANE* [36] It is an extended version of RTM which specifically trains a latent low-dimensional vector to represent each document node. The node structure, text content and latent topics are preserved in the final network representations.
- *TriDNR* [41] This method simultaneously exploits network structure, node content and label information for node representation learning. It is based on the as-sumption that network structures and contents can enhance each other to collectively characterize the affinities between nodes.
- *TWSRL* [29] It is a topic-aware Web service representation learning, which encodes both network content and structure for unified representation learning.
- *ARGA* [42] It is a graph autoencoder model where node embeddings are trained to reconstruct the graph structure. This method can preserve both network structure and node content in the node embeddings.
- $WSNE_{nolink}$ It is an variant of our proposed approach without incorporating the link prediction step. It learns Web service representations based on the propsoed Service-GCN model in this paper.
- *WSNE* It is our proposed approach, which first performs link prediction between Mashup and API services based on Att-RTM, and then learn service representation learning based on Service-GCN.

5.3 Experimental Setup

5.3.1 Evaluation Tasks

After obtaining the vector representations of Web services based on the entire Web service network (refer to Section 5.1), three downstream tasks are adopted to evaluate the quality of learned vector representations:

- *Supervised Service Classification:* It is a popular task [12], [30] to evaluate the network embedding methods, which aims to classify Web services into different functional categories based on the learned representations.
- *Unsupervised Service Clustering:* It is a popular task [2], [3] which aims to group Mashup or API services to respective functional categories. We first obtain the embedding vectors of Mashup and API services, and then perform the unsupervised clustering based on the widely used K-means algorithm.
- *Embedding Vector Visualization:* To obtain a straightforward comparison of Web service embedding

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:09:54 UTC from IEEE Xplore. Restrictions apply.

TABLE 2
Distribution of Web Services in Top 20 Categories

Category	Number	Category	Number
c_1: *Tools*	841	c_{11}: *Video*	412
c_2: *eCommerce*	740	c_{12}: *Telephony*	395
c_3: *Social*	739	c_{13}: *Reference*	392
c_4: *Financial*	632	c_{14}: *Payment*	385
c_5: *Search*	570	c_{15}: *Government*	385
c_6: *Messaging*	528	c_{16}: *Science*	304
c_7: *Enterprise*	511	c_{17}: *Sports*	265
c_8: *Photos*	459	c_{18}: *Advertising*	265
c_9: *Music*	440	c_{19}: *Education*	264
c_{10}: *Travel*	419	c_{20}: *Email*	262

results, we visualize service embedding vectors in a two-dimensional space based on t-SNE [43]. Normally, Web services from identical functional domains are represented in similar vector representations, thereby should be closed in the two-dimensional space.

Since the numbers of Web services in different functional categories are highly skewed, the overall classification and clustering performances would be significantly affected by categories that contain very few numbers of services. Because the algorithm would bias the training of majority service categories while leaving the minority service categories undertrained [44]. We therefore choose top 20 categories shown in Table 2 to evaluate all baseline methods.

5.3.2 Evaluation Metrics

WSNE performs a supervised service classification, while for other methods we first build a Support Vector Machine classifier with linear kernel [45] on the training data and then predict labels for all nodes in the test data. The performance is evaluated by two widely used metrics [12], [30]

$$\text{Micro} - \text{F1} = \frac{\sum_{i=1}^{|\mathbf{L}|} 2TP^i}{\sum_{i=1}^{|\mathbf{L}|} (2TP^i + FP^i + FN^i)} \tag{16}$$

$$\text{Macro} - \text{F1} = \frac{1}{|\mathbf{L}|} \sum_{i=1}^{|\mathbf{L}|} \frac{2TP^i}{(2TP^i + FP^i + FN^i)}, \tag{17}$$

where $|\mathbf{L}|$ indicates the total number of labels or categories, TP^i, FN^i and FP^i denote the number of true positive, false negative and false positive w.r.t the ith category, respectively.

For the unsupervised clustering task, K-mean algorithm is used to cluster the learned service representations of various baselines. Similar to previous work [46], we adopt four widely used metrics to compare the clustering performance. They are accuracy (Acc), Precision, F1-measure (F1) and normalized mutual information (NMI). The larger of these measured values, the better quality of the learned service representations by the respective method.

5.3.3 Parameter Settings

There are many hyper parameters need to be given for our method either empirically or experimentally. To train the proposed Att-RTM model, following the previous experience [2], [18], the number of topics (T) is set as 100, parameters α, β and η are set as 0.1, 0.05 and 2.0, respectively. For supervised training of the Service-GCN model, we randomly sample $p\%$ of Web service nodes with functional categories known for training. The remaining nodes are split into validation and testing sets, where 10 percent are used for parameter optimization and 90 percent are used for testing. For comparison reason, default settings of rest parameters are as follows. The number (K) of API services predicted in the link prediction process is set as 1. We use the 1-layer Service-GCN as the default model. We also compare with the 2-layer Service-GCN, where the first-layer (r) and second-layer (d) embedding dimensions are set as the total number of service functional categories involved in the input network. The training ratio (p) varies from 10 to 70 percent, where for each ratio the model is trained 400 times with SGD learning rate equal to 0.02. We use dropout and regularization techniques to mitigate the overfitting issue, where the dropout rate and L2 norm weight decay for Service-GCN[1] are set as 0.3 and 3e-5, respectively. For each given ratio (p) of training samples, the experiment is repeated 20 times, where the average results and standard deviations are reported.

For other baseline methods, we use the suggested parameter settings in the literature where these methods are respectively proposed. Please refer to their implementation details by the provided links on the Github page[1].

5.4 Experimental Results

5.4.1 Web Service Classification Results

For the Web service classification results shown in Table 3, we have the following major observations:

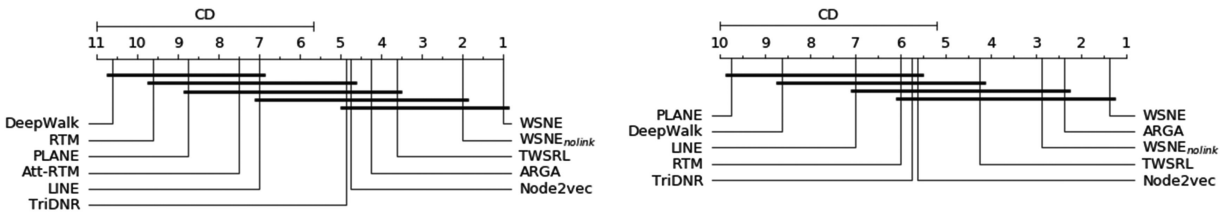
1) In general, content-preserved methods such as TriDNR, TWSRL and WSNE outperform methods only preserving the network structure such as DeepWalk and LINE. This is because node contents are helpful information to characterize affinities between nodes, especially when the network connectivity is sparse [36], [37]. Yet, node content for the Web service network may contain noise which may result in suboptimal embedding results. For example, the performance of RTM-based methods such as RTM and PLANE are relatively poorer compared to Att-RTM as they treat all word features (e.g., both functional words and noisy words) equally in the topic training and service similarity calculation.
2) Among all structure-based methods (i.e., DeepWalk, LINE and Node2vec), Node2vec performs the best in service representation learning observed in Table 3. The reason is that Node2vec adopts a more flexible neighborhood sampling strategy which interpolates between breadth-first sampling and depth-first sampling to capture functional relationships between services. From Table 3 we can also observe that Att-RTM significantly outperforms the basic RTM model. This is because service functional

1. https://github.com/shiminxst/wsne.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:09:54 UTC from IEEE Xplore. Restrictions apply.

TABLE 3
Classification Results for Various Training Ratios on Webservice (The Best Method is Marked With * If it is Significantly Better Than Others, Based on a *Student*'s t-test With p-value$\leq$0.05)

Metrics	Macro-F1				Micro-F1			
Train Ratio ($p\%$)	10	30	50	70	10	30	50	70
DeepWalk	0.079±0.004	0.101±0.006	0.112±0.007	0.115±0.007	0.147±0.008	0.184±0.006	0.209±0.008	0.225±0.010
LINE	0.200±0.011	0.255±0.013	0.274±0.012	0.283±0.010	0.291±0.012	0.347±0.009	0.373±0.012	0.388±0.012
Node2vec	0.232±0.016	0.297±0.010	0.323±0.013	0.336±0.014	0.323±0.014	0.392±0.006	0.429±0.009	0.446±0.014
RTM	0.109±0.011	0.157±0.011	0.175±0.014	0.192±0.012	0.161±0.010	0.217±0.011	0.244±0.011	0.260±0.015
PLANE	0.146±0.011	0.173±0.011	0.178±0.012	0.192±0.017	0.200±0.008	0.236±0.006	0.251±0.008	0.261±0.018
Att-RTM	0.207±0.016	*0.223*±0.006	0.229±0.008	0.232±0.017	0.303±0.009	0.325±0.008	*0.328*±0.008	0.341±0.014
TriDNR	0.219±0.021	0.265±0.011	0.283±0.010	0.296±0.010	0.390±0.015	0.430±0.008	0.444±0.009	0.450±0.009
TWSRL	0.232±0.013	0.286±0.009	0.312±0.008	0.336±0.007	0.424±0.008	0.460±0.007	0.466±0.006	0.475±0.005
ARGA	0.236±0.011	0.295±0.009	0.280±0.013	0.350±0.014	0.389±0.014	0.439±0.010	0.437±0.013	0.461±0.014
WSNE$_{nolink}$	0.359±0.001	0.405±0.001	0.445±0.004	0.448±0.005	0.479±0.001	0.543±0.002	0.551±0.003	0.556±0.002
WSNE	**0.378**±0.003*	**0.438**±0.001*	**0.476**±0.004*	**0.478**±0.003*	**0.480**±0.003*	**0.553**±0.002*	**0.580**±0.002*	**0.590**±0.001*



(a) Results of Nemenyi test on Webservice (CD=5.33)

(b) Results of Nemenyi test on Wiki (CD=4.79)

Fig. 6. Mean rank (the smaller, the better) and critical difference (CD) of the Nemenyi test for various methods. A method is considered as significantly different from another if its mean rank is larger by more than the respective CD. Groups of methods that are not significantly different at $p-$value = 0.05 are connected.

descriptions are littered with irrelevant or noisy word features. The attention mechanism introduced in Att-RTM can help to highlight functional-related words while learning topic-based representations of Mashup and API services.

3) We can observe that in most cases WSNE$_{nolink}$ achieves better results than TWSRL and TriDNR. There are two possible reasons. First, although all three methods have encoded service structure and content information, WSNE$_{nolink}$ is more efficient than TriDNR and TWSRL in feature learning as it adopts a supervised training which can automatically extract important features aligned with the classification task. Second, WSNE$_{nolink}$ adopts a convolution learning paradigm for all nodes on the network which allows both API and Mashup services encode first-order composition relationships for efficient representation learning. In contrast, TWSRL adopts a translation-based learning where only Mashup services can encode the composition relationships for representation learning. Compared with WSNE$_{nolink}$ that only models the first-order composition relationships, TriDNR depends on a random walk process over the whole network to capture high-order relationships between services, which may introduce noises since two API services in a walk may have different functions and should be dissimilar in the embedding space. Furthermore, the proposed method WSNE improved greatly as it can be observed in Table 3. This demonstrates that predicting potential composition links between API and Mashup services is helpful to enrich the service network connectivity and enhance the service representation learning.

We further perform a Friedman-Nemenyi test [47] for comparing various groups of methods across the average Macro-F1 and Micro-F1 results at different training ratios in Table 3. In the experiment, the Friedman null hypothesis is rejected with $p-$value=0.05. The calculated $p-$value for various methods in Table 3 is 1.4E-13, meaning that the outcome of different embedding methods is different. Then, we proceed with a post-hoc Nemenyi test to explore the significant difference between individual methods. We can observe from Fig. 6a that WSNE is only significant different from LINE, Att-RTM, PLANE, RTM and DeepWalk, where the corresponding common language effect sizes (CLES) [48][2] are 0.98, 1.0, 1.0, 1.0, 1.0, respectively. The differences between WSNE and other baseline methods including TWSRL, ARGA and Node2vec cannot be significantly discriminated by the Friedman-Nemenyi test. However, the *Student*'s $t-$test in Table 3 demonstrates that WSNE is significantly better than other baselines at various training ratios.

5.4.2 Web Service Clustering Results

We then compare all baseline methods by the unsupervised clustering task. For each baseline, the K-means algorithm is used to cluster learned service representations of services

2. CLES reflects the probability that a score sampled from one distribution will be grater than a score sampled from another distribution.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:09:54 UTC from IEEE Xplore. Restrictions apply.

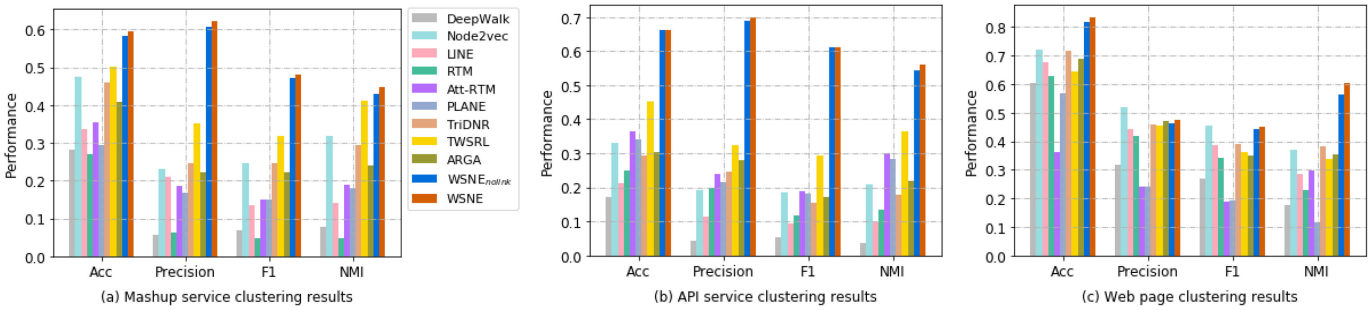


Fig. 7. Web service clustering results on Webservice data and Web page clustering results on Wiki data.

from the top 20 functional categories shown in Table 2. For the K-means algorithm, we set the number of clusters as 20 which corresponds to the 20 respective functional domains. Fig. 7a shows the Mashup service clustering performance, we can conclude that methods (e.g., WSNE, ARGA and TWSRL) with both network structure and content preserved perform generally better than methods (e.g., DeepWalk and LINE) preserving only the network structure, which is consistent with the observation from the service classification results. Att-RTM consistently outperforms RTM w.r.t all metrics which again demonstrates it is important to give higher attention to functional-related words instead of treating all words equally. The proposed WSNE model improves by a large margin compared to other methods, i.e., the average accuracy of WSNE improved 19.0 percent over TWSRL, which verifies that the learned embedding vectors are more meaningful and effective to distinguish between different functional clusters compared to other methods. Similar observations can be drew from the API service clustering results in Fig. 7b. It is interesting to observe that WSNE achieves more distinct improvement in API service clustering than in Mashup service clustering. The reason might be that APIs have less complicated functions than Mashups, thus it is much easy to learn distinguishing representations for API services in different functional domains.

5.4.3 Web Page Classification and Clustering Results

We additionally evaluate WSNE using a Web page network data (e.g., similar to the Web service network) by conducting supervised Web page classification and unsupervised Web page clustering. For classification results in Table 4, we can observe that WSNE is not significantly better compared with other baselines such as RTM (e.g., when p equals to 10 percent) and ARGA (e.g., when p equals to 10 or 50 percent) regarding Macro-F1. Nevertheless, the Micro-F1 performance of WSNE consistently outperforms other methods, i.e., the average Micro-F1 improved 8.4 and 13.3 percent compared to ARGA and TWSRL respectively. The distinct advantage of WSNE compared with other methods is threefold: 1) The predicted potential linkage relationships between nodes in the network are helpful for enriched and improved node representation learning; 2) The convolutional learning based on GCN is efficient to capture relationships between nodes in the network; 3) WSNE adopts an end-to-end supervised training which benefits from the label information to efficiently learn meaningful node representations, i.e., nodes with identical labels are mapped in similar representations. Fig. 6b shows the Nemenyi test results for various groups of methods and their classification results in Table 4. We can observe that WSNE is significantly different compared with LINE, DeepWalk and PLANE, where the common language effect sizes are 0.56, 0.94 and 1.0, respectively. In addition, the *Student*'s $t-$test in Table 4 demonstrate that WSNE is significantly better than all other baseline methods *w.r.t* the Micro-F1 results.

For Web page clustering results in Fig. 7c, we can conclude two major observations as follows. First, WSNE generally excels regarding all four evaluation metrics compared to other methods except Node2vec. Second, WSNE is not significantly different from Node2vec w.r.t F1 (a comprehensive measure of Acc and Precision). But WSNE has better NMI performance

TABLE 4
Classification Results for Various Training Ratios on Wiki (The Best Method is Marked With * If it is Significantly Better Than Others, Based on a *Student*'s t-test With p-value$\leq$0.05)

Metrics	Macro-F1				Micro-F1			
Train Ratio (p%)	10	30	50	70	10	30	50	70
DeepWalk	0.216±0.018	0.274±0.008	0.295±0.011	0.302±0.021	0.322±0.014	0.397±0.010	0.427±0.011	0.443±0.013
LINE	0.344±0.019	0.431±0.016	0.459±0.024	0.482±0.029	0.443±0.016	0.536±0.010	0.566±0.015	0.582±0.017
Node2vec	0.381±0.013	0.455±0.018	0.494±0.019	0.512±0.022	0.468±0.017	0.568±0.010	0.607±0.009	0.628±0.013
RTM	**0.448**±0.027	0.468±0.047	0.418±0.043	0.372±0.042	0.554±0.026	0.573±0.052	0.473±0.039	0.435±0.040
PLANE	0.161±0.013	0.215±0.007	0.235±0.010	0.204±0.010	0.241±0.013	0.309±0.009	0.338±0.010	0.364±0.014
TriDNR	0.385±0.016	0.441±0.013	0.452±0.012	0.479±0.016	0.564±0.011	0.606±0.010	0.616±0.012	0.628±0.012
TWSRL	0.415±0.012	0.463±0.013	0.472±0.013	0.486±0.016	0.578±0.012	0.631±0.010	0.643±0.011	0.656±0.013
ARGA	0.425±0.012	0.497±0.019	**0.516**±0.011	**0.571**±0.015*	0.583±0.016	0.675±0.012	0.679±0.014	0.683±0.018
WSNE$_{nolink}$	0.405±0.018	0.463±0.018	0.503±0.023	0.529±0.026	0.652±0.012	0.712±0.010	0.728±0.012	0.724±0.017
WSNE	0.437±0.005	**0.504**±0.007	0.514±0.009	0.536±0.003	**0.661**±0.008*	**0.720**±0.007*	**0.730**±0.005*	**0.731**±0.005*

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:09:54 UTC from IEEE Xplore. Restrictions apply.

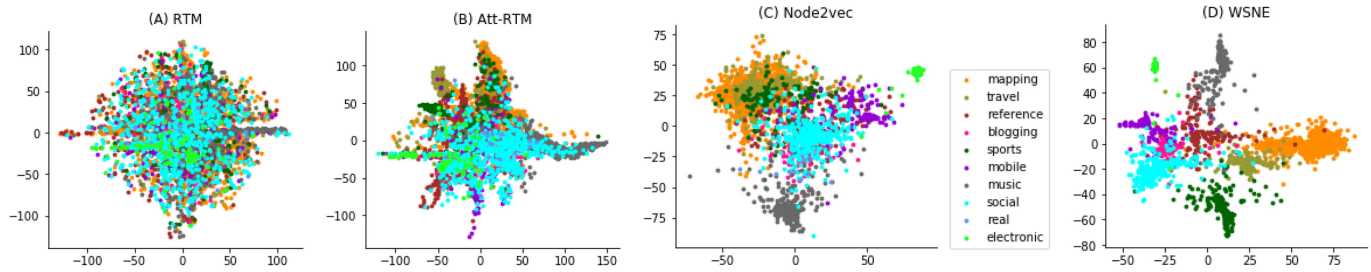


Fig. 8. Visualization of service embedding vectors learned by different methods. Each data point represents a service and services in the same functional category are marked with the same color.

than Node2vec, which means WSNE can produce much distinguishing boundaries between clusters.

5.4.4 Web Service Embedding Vector Visualization

We visualize low-dimensional vectors of services in the two-dimensional space to obtain an intuitive observation of the embedding results (refer to Section 5.3.1). The visualization results of RTM, Att-RTM, Node2vec and WSNE w.r.t the selected 10 functional categories are shown in Fig. 8, where x and y axes represent positions of services in the euclidean space. The proposed Att-RTM model generates more meaningful representations of services than RTM, i.e., services in the category of *social* are more compact to each other. This is because RTM simply treat words in service text content equally, whereas Att-RTM can allocate more attention to the functional-related words than irrelevant words which helps to learn improved topic representations of services. In addition, we can also observe that Node2vec and WSNE produce more meaningful visualization results than Att-RTM and RTM, which are consistent with the comparisons observed in service classification (Table 3) and service clustering (Figs. 7a and 7b). WSNE presents more distinguishing boundaries between services in different functional categories, which demonstrates the effectiveness of the proposed model in learning meaningful low-dimensional Web service representations.

5.4.5 Impacts of Convolutional Layers and Predicted Links

We compare WSNE models with 1-layer and 2-layer Service-GCNs, respectively. We can observe from Fig. 9a that the 1-layer model is superior to the 2-layer model on both Macro-F1 and Micro-F1. The main reason is that while the 2-layer model allows to preserve second-order relationships between services in the embedding space, it meanwhile enforces irrelevant feature propagation between API services on the network because member APIs composed in a specific Mashup often present different functional properties. In other words, different member API nodes (e.g., node 1 and node 5 in Fig. 5) linking to the same Mashup are dissimilar and feature sharing between them may deteriorate the relationships between services. Fig. 9b shows the impact of the number (K) of API services predicted for each Mashup service. We can observe that larger values of K after 1 tends to produce poor performance.

6 Discussion

Low-dimensional vector representation of Web services are fundamental for many downstream applications [29] including Web service functionality classification and functionality clustering, which provides an efficient way to compare functional similarities between Web services. We argue that it is necessary to preserve both network structure and content information in the vector representations of Web services. However, the convolutional learning adopted in this paper relies on rich structure links to capture similarities between services, which motivated us to predict potential network links based on the similarities between service text contents. We perform downstream classification and clustering tasks to validate the quality of learned vector representations. The evaluation results in different Web service categories vary significantly shown in Fig. 9c. Recent studies [44] shows that when multiple categories present imbalanced numbers of samples, the overall results may be affected by categories containing very few number of samples. We therefore choose top 20 categories that contain the majority of services for evaluating all baseline methods.

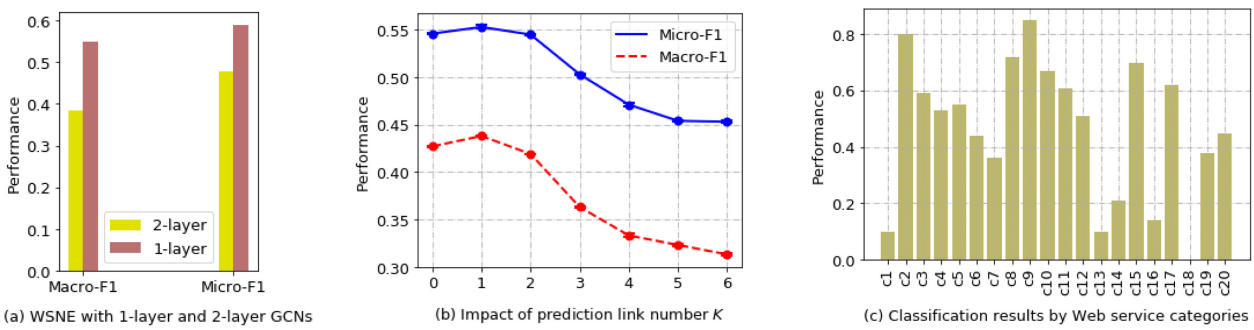


Fig. 9. (a) compares the 1-layer and 2-layer Service-GCNs, (b) tests the influence of the number of predicted links K, and (c) shows the classification results by Web service categories.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:09:54 UTC from IEEE Xplore. Restrictions apply.

From the comparisons, we conclude that WSNE$_{nolink}$, TWSRL and TriDNR significantly outperform other content-centered methods, which answered the first research question (refer to Section 3.1) that network-centered methods are effective to learn meaningful service representations. The second research question (refer to Section 3.1) was addressed by the observation that WSNE$_{nolink}$ with supervised embedding learning is superior to both network-centered and content-centered baselines with unsupervised training. Finally, we observe that WSNE achieves a large performance gain over WSNE$_{nolink}$, which answered the third research question (refer to Section 3.1) proposed in this paper.

Although the comparison in Fig. 9a shows that it is rational to adopt the 1-layer Service-GCN only preserving the first-order structure relationships, we argue it is promising to explore high-order correlations between services. For example, two Mashup services could be similar if they have some common member API services, where their functional similarities can be modeled using a 2-layer Service-GCN.

7 Conclusion

In this paper, we studied a Web service network embedding problem, which aims to represent Web services with low-dimensional vectors by preserving both network structure and content information. We analyzed the necessity of predicting the missing or potential composition links between Web services. To improve the prediction accuracy, we proposed a functional attention-based probabilistic model, Att-RTM, which can highlight functional-related words while inferring the latent topics of services. To perform service embedding learning, we developed Service-GCN to efficiently capture service text content and structural links for supervised vector representation learning. The Att-RTM model can be trained offline and the complexity of Service-GCN is the same as that of GCN which is $O(|\mathbf{E}|d)$, where $|\mathbf{E}|$ is the total number of edges on the network and d is the dimension of the service representations.

We designed experiments on two real-world datasets to evaluate the proposed approach. From the results, we can conclude that 1) Web service text contents may contain noisy words and it is helpful to highlight functional-oriented words; 2) The Att-RTM model is effective to elicit latent topics; 3) Our deep framework for service representation significantly outperforms the state-of-the-art methods.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China under Grant 61872139 and in part by National Science Foundation under Grants IIS-1763452 and CNS-1828181.

References

[1] L. Baresi, E. Di Nitto, and C. Ghezzi, "Toward open-world software: Issues and challenges," *Computer*, vol. 39, no. 10, pp. 36–43, Oct. 2006.

[2] M. Shi, J. Liu, D. Zhou, M. Tang, and B. Cao, "We-LDA: A word embeddings augmented LDA model for web services clustering," in *Proc. IEEE Int. Conf. Web Serv.*, 2017, pp. 9–16.

[3] B. Cao, X. Liu, M. M. Rahman, B. Li, J. Liu, and M. Tang, "Integrated content and network-based service clustering and web APIS recommendation for mashup development," *IEEE Trans. Serv. Comput.*, vol. 13, no. 1, pp. 99–113, Jan./Feb. 2020.

[4] B. Cheng, S. Zhao, C. Li, and J. Chen, "A web services discovery approach based on mining underlying interface semantics," *IEEE Trans. Knowl. Data Eng.*, vol. 29, no. 5, pp. 950–962, May 2017.

[5] W. Chen, I. Paik, and P. C. Hung, "Constructing a global social service network for better quality of web service discovery," *IEEE Trans. Serv. Comput.*, vol. 8, no. 2, pp. 284–298, Mar./Apr. 2015.

[6] Q. Gu, J. Cao, and Q. Peng, "Service package recommendation for mashup creation via mashup textual description mining," in *Proc. IEEE Int. Conf. Web Serv.*, 2016, pp. 452–459.

[7] M. Garriga, C. Mateos, A. Flores, A. Cechich, and A. Zunino, "Restful service composition at a glance: A survey," *J. Netw. Comput. Appl.*, vol. 60, pp. 32–53, 2016.

[8] W. Shi, X. Liu, and Q. Yu, "Correlation-aware multi-label active learning for web service tag recommendation," in *Proc. IEEE Int. Conf. Web Serv.*, 2017, pp. 229–236.

[9] M. Shi, J. Liu, D. Zhou, and Y. Tang, "A topic-sensitive method for mashup tag recommendation utilizing multi-relational service data," *IEEE Trans. Serv. Comput.*, vol. 14, no. 2, pp. 342–355, Mar./Apr. 2021.

[10] J. Ma, Y. Zhang, and J. He, "Web services discovery based on latent semantic approach," in *Proc. IEEE Int. Conf. Web Serv.*, 2008, pp. 740–747.

[11] J. Li, C. Chen, H. Tong, and H. Liu, "Multi-layered network embedding," in *Proc. SIAM Int. Conf. Data Mining*, 2018, pp. 684–692.

[12] A. Grover and J. Leskovec, "NODE2VEC: Scalable feature learning for networks," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 855–864.

[13] D. Zhang, J. Yin, X. Zhu, and C. Zhang, "Network representation learning: A survey," *IEEE Trans. Big Data*, vol. 6, no. 1, pp. 3–28, Mar. 2020.

[14] I. Ahmad, M. U. Akhtar, S. Noor, and A. Shahnaz, "Missing link prediction using common neighbor and centrality based parameterized algorithm," *Sci. Rep.*, vol. 10, no. 1, pp. 1–9, 2020.

[15] D. You, N. Vo, K. Lee, and Q. Liu, "Attributed multi-relational attention network for fact-checking URL recommendation," in *Proc. 28th ACM Int. Conf. Inf. Knowl. Manage.*, 2019, pp. 1471–1480.

[16] X. Si and M. Sun, "Tag-LDA for scalable real-time tag recommendation," *J. Comput. Inf. Syst.*, vol. 6, no. 1, pp. 23–31, 2009.

[17] Y. Xiao, J. Liu, R. Hu, B. Cao, and Y. Cao, "DINREC: Deep interest network based API recommendation approach for mashup creation," in *Proc. Int. Conf. Web Inf. Syst. Eng.*, 2019, pp. 179–193.

[18] L. Ren and W. Wang, "An SVM-based collaborative filtering approach for top-n web services recommendation," *Future Gener. Comput. Syst.*, vol. 78, pp. 531–543, 2018.

[19] C. Platzer and S. Dustdar, "A vector space search engine for web services," in *Proc. 3rd Eur. Conf. Web Serv.*, 2005, pp. 62–71.

[20] M. Shi, J. Liu, D. Zhou, M. Tang, F. Xie, and T. Zhang, "A probabilistic topic model for mashup tag recommendation," in *Proc. IEEE Int. Conf. Web Serv.*, 2016, pp. 444–451.

[21] B. Cao, J. Liu, Y. Wen, H. Li, Q. Xiao, and J. Chen, "QOS-aware service recommendation based on relational topic model and factorization machines for IoT mashup applications," *J. Parallel Distrib. Comput.*, vol. 132, pp. 177–189, 2019.

[22] G. Cassar, P. M. Barnaghi, and K. Moessner, "Probabilistic methods for service clustering," in *Proc. SMRR@ ISWC*, 2010, pp. 4–20.

[23] X. Yan, J. Guo, Y. Lan, and X. Cheng, "A biterm topic model for short texts," in *Proc. 22nd Int. Conf. World Wide Web*, 2013, pp. 1445–1456.

[24] Y. Zhong, Y. Fan, W. Tan, and J. Zhang, "Web service recommendation with reconstructed profile from mashup descriptions," *IEEE Trans. Autom. Sci. Eng.*, vol. 15, no. 2, pp. 468–478, Apr. 2018.

[25] C. Li, R. Zhang, J. Huai, and H. Sun, "A novel approach for API recommendation in mashup development," in *Proc. IEEE Int. Conf. Web Serv.*, 2014, pp. 289–296.

[26] L. Liao, X. He, H. Zhang, and T.-S. Chua, "Attributed social network embedding," *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 12, pp. 2257–2270, Dec. 2018.

[27] J. Chang and D. Blei, "Relational topic models for document networks," in *Proc. Artif. Intell. Statist.*, 2009, pp. 81–88.

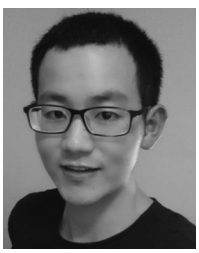
[28] O. Adeleye, J. Yu, J. Ruan, and Q. Z. Sheng, "Evaluating random walk-based network embeddings for web service applications," in *Proc. Australas. Database Conf.*, 2020, pp. 198–205.

[29] M. Shi, Y. Tang, X. Zhu, and J. Liu, "Topic-aware web service representation learning," *ACM Trans. Web*, vol. 14, no. 2, pp. 1–23, 2020.

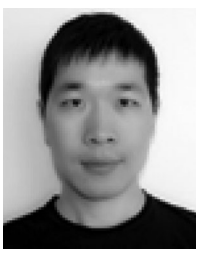
[30] B. Perozzi, R. Al-Rfou, and S. Skiena, "Deepwalk: Online learning of social representations," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2014, pp. 701–710.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:09:54 UTC from IEEE Xplore. Restrictions apply.

[31] X. Huang, J. Li, and X. Hu, "Label informed attributed network embedding," in *Proc. 10th ACM Int. Conf. Web Search Data Mining*, 2017, pp. 731–739.
[32] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," 2013, *arXiv:1301.3781*.
[33] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, and Q. Mei, "Line: Large-scale information network embedding," in *Proc. 24th Int. Conf. World Wide Web*, 2015, pp. 1067–1077.
[34] C. Tu, W. Zhang, Z. Liu, and M. Sun, "Max-margin deepwalk: Discriminative learning of network representation," in *Proc. 25h Int. Joint Conf. Artif. Intell.*, 2016, pp. 3889–3895.
[35] J. Chen, Q. Zhang, and X. Huang, "Incorporate group information to enhance network embedding," in *Proc. 25th ACM Int. Conf. Inf. Knowl. Manage.*, 2016, pp. 1901–1904.
[36] T. M. Le and H. W. Lauw, "Probabilistic latent document network embedding," in *Proc. IEEE Int. Conf. Data Mining*, 2014, pp. 270–279.
[37] C. Yang, Z. Liu, D. Zhao, M. Sun, and E. Chang, "Network representation learning with rich text information," in *Proc. 24th Int. Joint Conf. Artif. Intell.*, 2015.
[38] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.
[39] C. Andrieu, N. De Freitas, A. Doucet, and M. I. Jordan, "An introduction to MCMC for machine learning," *Mach. Learn.*, vol. 50, no. 1–2, pp. 5–43, 2003.
[40] L. Bottou, "Large-scale machine learning with stochastic gradient descent," in *Proc. COMPSTAT.*, 2010, pp. 177–186.
[41] S. Pan, J. Wu, X. Zhu, C. Zhang, and Y. Wang, "Tri-party deep network representation," *Network*, vol. 11, no. 9, 2016, Art. no. 12.
[42] S. Pan, R. Hu, G. Long, J. Jiang, L. Yao, and C. Zhang, "Adversarially regularized graph autoencoder for graph embedding," 2018, *arXiv:1802.04407*.
[43] L. V. D. Maaten and G. Hinton, "Visualizing data using T-SNE," *J. Mach. Learn. Res.*, vol. 9, no. 86, pp. 2579–2605, 2008.
[44] M. Shi, Y. Tang, X. Zhu, D. Wilson, and J. Liu, "Multi-class imbalanced graph convolutional network learning," in *Proc. 29th Int. Joint Conf. Artif. Intell.*, 2020, pp. 2879–2885.
[45] A. Abraham *et al.*, "Machine learning for neuroimaging with scikit-learn," *Frontiers Neuroinf.*, vol. 8, 2014, Art. no. 14.
[46] R. Xia, Y. Pan, L. Du, and J. Yin, "Robust multi-view spectral clustering via low-rank and sparse decomposition," in *Proc. 28th AAAI Conf. Artif. Intell.*, 2014, pp. 2149–2155.
[47] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *J. Mach. Learn. Res.*, vol. 7, pp. 1–30, 2006.
[48] K. O. McGraw and S. P. Wong, "A common language effect size statistic," *Psychol. Bulletin*, vol. 111, no. 2, 1992, Art. no. 361.



Min Shi (Student Member, IEEE) received the MS degree from the School of Computer Science and Engineering, Hunan University of Science and Technology, Xiangtan, China, and the PhD degree in computer science from Florida Atlantic University, FL, USA. He is currently a postdoc research associate with the School of Medicine, Washington University, St. Louis, MO, USA. His research interests include data mining, machine learning, graph mining, bioinformatics, and service computing.



Yuan Zhuang (Member, IEEE) received the bachelor's degree in information engineering and the master's degree in microelectronics and solid-state electronics from Southeast University, Nanjing, China, in 2008 and 2011, respectively, and the PhD degree in geomatics engineering from the University of Calgary, Canada, in 2015. Since 2015, he has been a lead scientist with Bluvision Inc. (acquired by HID Global), Fort Lauderdale, FL, USA. He has coauthored more than 50 academic papers and 11 patents. His current research interests include real-time location system, personal navigation system, wireless positioning, multi-sensors integration, Internet of Things , and machine learning for navigation applications. He is an associate editor for *IEEE Access*, the guest editor of the *IEEE Internet of Things Journal* and the *IEEE Access*, and a reviewer of more than ten IEEE journals. He was the recipient of more than ten academic awards.

Yufei Tang (Member, IEEE) received the BEng and MEng degrees from Hohai University, Nanjing, China, in 2008 and 2011, respectively, and the PhD degree in electrical engineering from the Department of Electrical, Computer, and Biomedical Engineering, University of Rhode Island, Kingston, RI, USA, in 2016. He is currently an assistant professor with the Department of Electrical Engineering and Computer Science and a faculty fellow with the Institute for Sensing and Embedded Network Systems Engineering, Florida Atlantic University, Boca Raton, FL, USA. His research interests include computational intelligence, big data, smart grid, and cyber-physical systems.

Maohua Lin received the BEng degree from Fuzhou University, Fuzhou, China in 2007, the MEng degree from Nanchang University, Nanchang, China, in 2013, respectively, and the PhD degree in mechanical engineering from Florida Atlantic University, Boca Raton, Fl, USA, in 2019. He is currently a research scientist with Florida Atlantic University, Boca Raton, Fl, USA. His research interests include computational model, machine learning, sensor design, and medical device.

Xingquan Zhu (Senior Member, IEEE) received the PhD degree in computer science from Fudan University, Shanghai, China. He is currently a professor with the Department of Electrical Engineering and Computer Science, Florida Atlantic University, Boca Raton, FL, USA. He has authored or coauthored more than 280 refereed journal and conference papers in the areas of his research interests. His research interests include data mining, machine learning, multimedia computing, and bioinformatics. He is an associate editor for the *IEEE Transactions On Knowledge And Data Engineering* from 2008 to 2012 and from 2014 till date. Since 2017, he has been an associate editor for the *ACM Transactions On Knowledge Discovery From Data*. He was the recipient of four Best Paper Awards and three Best Student Paper Awards.

Jianxun Liu received the MS degree in computer science from the Central South University of Technology in 1997 and the PhD degree in computer science from Shanghai Jiao Tong University in 2003. He is currently a professor and the dean with the School of Computer Science and Engineering, Hunan University of Science and Technology. His research interests include service computing, cloud computing, and big data.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/csdl.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:09:54 UTC from IEEE Xplore. Restrictions apply.